

Software: A software is a collection of computed programs that helps us to perform a task.

Types of softwares:

1. System Software

System software helps the computer to work and manages the hardware.

Examples:

- Windows
 - Linux
-

2. Application Software

Application software is used by users to do specific tasks.

Examples:

- WhatsApp
 - Chrome
 - MS Word
-

3. Programming Software

Programming software is used by developers to write and develop programs.

Examples:

- Compiler
- Visual Studio
- Eclipse

What is software testing?

“Software Testing is a part of the software development process used to detect and identify defects or bugs in the software. The main objective of software testing is to ensure the application is working correctly and to deliver a quality product to the client.”

Quality Software

1. Bug free
2. Meets requirements and or expectations
3. Delivered on time
4. Within budget

4.maintainable

Project vs product

- **Project:**
“If a software application is developed for a specific customer based on their requirements, then it is called a Project.”
- **Product:**
“If a software application is developed for multiple customers based on market needs or common requirements, then it is called a Product.”

Example

- **Project:** Banking software developed only for one bank.
- **Product:** Microsoft Word used by many customers worldwide.

Error vs Bug Vs Failure

- “An error is a human mistake made during software development. It can occur due to wrong coding, incorrect logic, misunderstanding requirements, or design issues.
- A bug is a defect in the software caused by an error, which leads to incorrect or unexpected behavior. Bugs are usually identified during testing.
- A failure occurs when the software does not perform the expected functionality during execution or runtime.”

Why the software has bugs?

- Miscommunication or no communication
- Software complexity
- Programming errors
- Changing requirements
- Lack of skilled testers

SDLC

It is step by step process used by the software industry to design , develop and test software.

3 main p's

P=people

P=process

P=product

SDLC Phases

1. **Requirement Gathering & Analysis**
Collect and analyze customer requirements.
2. **Planning**
Estimate time, cost, resources, and schedule.
3. **Design**
Create system architecture and software design.

4. **Development / Coding**
Developers write the code.
5. **Testing**
Test the application to find bugs and verify functionality.
6. **Deployment**
Release the software to the client or production.
7. **Maintenance**
Fix issues and provide updates after release.

Waterfall Model

The Waterfall Model is a software development life cycle model in which the development process flows step by step in a sequential order. In this model, each phase must be completed before moving to the next phase. The output of one phase becomes the input for the next phase.

It is called the “Waterfall Model” because the process flows downward like a waterfall.

Phases of Waterfall Model

1. Requirement Analysis

Customer requirements are collected and analyzed.

2. System Design

System architecture and software design are prepared based on requirements.

3. Development / Coding

Developers write the code for the application.

4. Testing

The application is tested to identify defects and ensure quality.

5. Deployment

The software is released to the client or production environment.

6. Maintenance

Bugs are fixed and updates are provided after deployment.

Advantages

- Simple and easy to understand

- Easy to manage
- Suitable for small projects with fixed requirements

Disadvantages of Waterfall Model

- Requirement changes are difficult once development starts.
- Testing is done only after the development phase.
- Not suitable for large and complex projects.
- Customer involvement is less after requirement gathering.
- Takes more time to deliver the final product.

The Spiral Model is an iterative SDLC model in which the software is developed step by step through repeated cycles called spirals.

Each spiral goes through phases such as planning, risk analysis, development, and testing.

The process continues until the final product is completed.

The Spiral Model mainly focuses on risk analysis and is mostly used for large, complex, and high-risk projects.

Phases of Spiral Model

1. Planning

Requirements are collected, analyzed, and project planning is done.

2. Risk Analysis

Possible risks are identified and solutions are analyzed to reduce project risk.

3. Development / Engineering

Coding and development of the software are performed.

4. Testing & Evaluation

The developed module is tested and customer feedback is collected.

After completion of one spiral, the same process repeats for the next iteration.

Spiral Model

Spiral Model is iterative model.

Spiral Model overcome drawbacks of Waterfall model.

We follow spiral model whenever there is dependency on the modules.

In every cycle new software will be released to customer.

Software will be released in multiple versions. So it is also called version control model.

Advantages of Spiral Model

- Testing is done in every cycle, before going to the next cycle.
- Customer will get to use the software for every module.
- Requirement changes are allowed after every cycle before going to the next cycle.

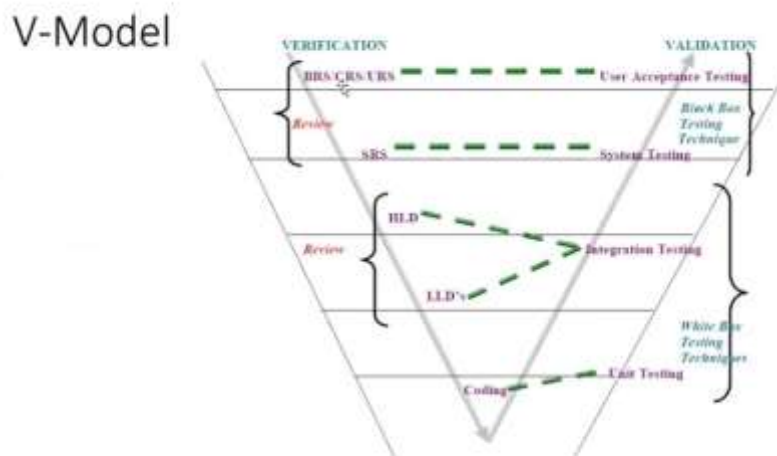
Disadvantages of Spiral Model

- Requirement changes are NOT allowed in between the cycle.
- Every cycle of spiral model looks like waterfall model.
- There is no testing in requirement & design phase.

V-Model

V-Model is a software development life cycle model in which testing activities are performed parallel to development activities.

In this model, for every development phase there is a corresponding testing phase.



It is called V-Model because the execution flow appears in the shape of the letter “V”.

Phases of V-Model

Verification Phases

- Requirement Analysis
- System Design
- High Level Design
- Low Level Design
- Coding

Validation Phases

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

Verification

- Verification means checking whether we are developing the product correctly according to requirements.
It is process-oriented and focuses on documentation

Static Testing

Testing the project related documents is called static testing

Reviews

Walkthroughs

inspections.

- **Validation**
- Validation means checking whether the developed product is working correctly according to user needs.
It is product-oriented and focus on software

Dynamic Testing

Testing the software

1. Unit testing

2. Integration testing

3. System Testing

4. UAT testing

- “Verification checks whether we are developing the product correctly, while validation checks whether we developed the correct product.”

Advantages of v model

- Testing is involved in each and every phase
- Defects are identified early

- Easy to understand
- Easy to manage
- Gives quality product

Disadvantages of V-Model

- Documentation is more
- Requirement changes are difficult
- Not suitable for large projects
- Less flexible

Static Checking

Review

Review is the process of checking documents or code to identify mistakes and improve quality.

Walkthrough

Walkthrough is an informal review where the author explains the document or code step by step to the team members.

Inspection

Inspection is a formal review process where a team checks the software or documents to find defects using defined rules and procedures.

Simple interview style:

- Review → Checking work for defects
- Walkthrough → Author explains the work to team
- Inspection → Formal checking process to identify defects

Review

Conducts on documents to ensure correctness and completeness

Requirement Reviews, Design Reviews, Code Reviews, Test plan reviews, Test cases reviews etc

Walkthrough

It is a informal review

Author reads the documents or code and discuss with peers.

It's not pre-planned and can be done whenever required.

Inspection

Its a most formal review type.

In which at least 3-8 people will sit in the meeting.

- 1- Reader
- 2- Writer
- 3- Moderator plus concerned people.

Inspection will have a proper schedule which will be intimated via email to the concerned developer/tester.

QA (Quality Assurance)

Quality Assurance is a process of ensuring that proper methods, standards, and processes are followed to develop a quality product.

It is process-oriented and focuses on preventing defects.

QC (Quality Control)

Quality Control is a process of checking the software product to identify defects and ensure quality.

It is product-oriented and focuses on finding defects.

Simple Difference

QA → Prevents defects

QC → Finds defects

Interview Style Answer

“QA focuses on preventing defects in the process, whereas QC focuses on identifying defects in the product.”

qe def chepu manchiga

QE (Quality Engineering)

“Quality Engineering is a process of ensuring and improving software quality throughout the entire SDLC using testing, automation, and continuous improvement practices.”

QA → Prevent defects

QC → Find defects

QE → Build quality throughout SDLC

Levels of testing

1. Unit testing
2. Integration Testing
3. System testing
4. UAT

Unit testing

1. A unit is a single component or module of a software
2. Unit testing conducts on single program or single module
3. Unit testing is white box testing technique
4. Unit testing is conducted by the developers

Unit Test Techniques

1. Basic path testing
2. Control structure testing- a) Conditional testing b) Loops coverage
3. Mutation testing

Integration Testing

Integration testing performed between 2 or modules.

Integration testing focuses on checking data communication b/w multiple modules

It is white box testing technique

Integration Testing can be done by both **developers** and **testers** depending on the project and company process.

- **Developers** usually perform integration testing during development to check whether modules work together properly.
- **Testers** perform integration testing from the testing side to validate the integrated modules.

Types of Integration Testing

1. Incremental Integration Testing
2. Non-Incremental Integration Testing

Incremental Integration Testing

Incrementally adding the modules and testing the data flow between the modules.

Approaches

- Top-Down Approach
- Bottom-Up Approach
- Sandwich/Hybrid Approach

Top-Down Integration Testing

In Top-Down Integration Testing, testing starts from the main or top-level module and gradually moves towards the lower-level modules.

If lower-level modules are not developed, temporary programs called **stubs** are used.

Example

```
Module A
  ↓
Module B
  ↓
Module C
```

Testing starts from Module A and moves downward.

Bottom-Up Integration Testing

In Bottom-Up Integration Testing, testing starts from lower-level modules and gradually moves towards the top-level modules.

If top-level modules are not developed, temporary programs called **drivers** are used.

Example

```
Module C
  ↑
Module B
  ↑
Module A
```

Testing starts from Module C and moves upward.

Sandwich/Hybrid Integration testing

Combination of top down and bottom up

Non –Incremental integration Testing

Adding all the modules in a single shot and test the data flow b/w modules

Drawbacks:

- We might miss data flow between some of the modules
- If you find any defect we can't understand the root cause of defect

System Testing

Testing over all functionality of the application with respective client requirements

It is a black box testing technique

This testing is conducted by testing team.

After completion of component(unit) and integration level testing, we start system testing.

Before conducting system testing, we should know the customer requirements.

System Testing focuses on below aspects:

- User Interface Testing (GUI)
- Functional Testing
- Non-Functional Testing
- Usability Testing

User Acceptance Testing (UAT)

After completion of system testing, UAT team conducts acceptance testing in two levels:

- Alpha testing
- Beta testing

Alpha Testing

“Alpha testing is a type of testing performed by developers or testers inside the company before releasing the software to customers.”

Beta Testing

“Beta testing is a type of testing performed by real users or customers in the real environment before final product release.”

Unit test and integration test-white box

System test and uat-black box

System testing

1.GUI Testing

- GUI testing is a process of testing the user interface of an application
- A graphical user interface includes all the elements such as menus,checkbox,buttons,colors,fonts,sizes,icons,content and images

2.Usability Testing

- During this testing we check whether the application provides context sensitive help or not to the user
- Checks how easily the end users are able to understand and operate the application is called usability testing

3.Functional Testing

Functionality means behavior of the application

Functional Testing is a software testing technique used to verify whether the application works according to the specified requirements and customer expectations.”

4.Non-Functional Testing

Once the application functionality is stable then we do Non-functional Testing

Focus on performance,load it can take and security etc.

“Non-Functional Testing is a type of testing used to verify the non-functional aspects of an application such as performance, usability, reliability, security, and scalability.”

Regression testing

Regression Testing is a type of testing performed to verify that existing functionalities are not affected after new changes, bug fixes, or enhancements in the application.”

Types of Regression Testing

1. Unit Regression Testing

Testing only the modified module or component without affecting other modules.

2. Regional Regression Testing

Testing the modified module along with the impacted modules

Impact analysis meeting is conducted to identify related or impacted modules.

3. Full Regression Testing

Testing the complete application to ensure that new changes have not affected existing functionalities.

ReTesting

“Retesting is a type of testing performed to verify whether the reported defect is fixed by the developer or not.”

Retesting Example

A login button is not working properly.

Developer fixes the bug.

Tester tests the login button again to check whether the issue is fixed or not.

→ This is Retesting.

Unit Regression Testing Example

After fixing the login button bug, tester checks other functionalities in the login module like:

- Forgot Password
- OTP verification
- Password validation

to ensure the changes did not affect them.

→ This is Unit Regression Testing.

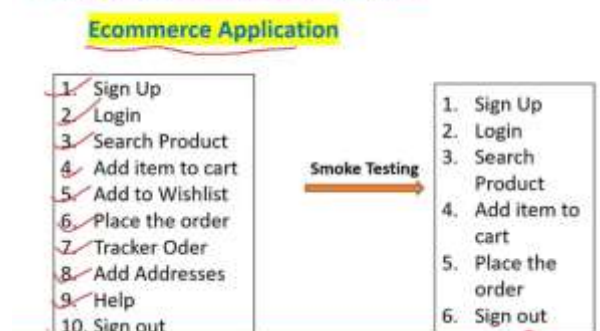
Smoke Testing

- Smoke Testing means testing the major or critical features of an application before going in detail testing.

- When ever we receive a new build from developers, we will first do smoke testing.

- It is also called as Build Verification Testing.

Smoke Testing Examples



Sanity Testing

Sanity Testing is a type of software testing performed to check whether the recent changes or bug fixes are working properly and the application is stable enough for further testing.

Interview Style Definition:

“Sanity Testing is a type of testing performed to verify that the newly added changes or bug fixes are working correctly and the related functionalities are not affected.”

Simple Example:

Suppose developers fixed the **login button issue**.
Tester will only check:

- Login is working or not
- Related pages are opening or not

They won't test the entire application deeply.

Important Points:

- Done after small changes or bug fixes
- Checks only related functionality
- Usually performed by testers
- It is a subset of Regression Testing
- Narrow and deep testing

Example:

Bug fixed → “Forgot Password not working”

Sanity testing checks:

- Forgot Password works properly
- OTP/mail is coming
- Password reset works

Not the whole application.

“Sanity Testing verifies whether the recent changes are working correctly, whereas Regression Testing verifies that existing functionalities are not affected due to new changes.”

Smoke Testing

v/s

Sanity Testing

- | | |
|---|---|
| <ul style="list-style-type: none">• Verifies <u>critical features</u> of an application.• <u>Wide and shallow testing</u>.• <u>Done by Developers and Testers</u>.• In smoke testing, build may be stable or unstable.• It is documented and scripted.• It is a subset of Acceptance testing.• It focuses on general health check up of an application. | <ul style="list-style-type: none">• Verifies <u>new functionality</u> and bug fixes.• <u>Deep and narrow testing</u>.• Done by Testers only.• Build is stable.• It is not documented and scripted.• It is a subset of Regression Testing.• It focuses on specialised health check up of an application. |
|---|---|

“Smoke Testing is performed to check whether the main functionalities of the application are working properly after receiving a new build, whereas Sanity Testing is performed to check whether the newly added changes or bug fixes are working correctly.”

Exploratory Testing

“Exploratory Testing is a type of testing in which the tester explores and understands the application completely, identifies possible scenarios, and tests the application without predefined requirements or test cases.”

Simple Points

- Tester explores the application and tests it.
- Performed when requirements are not available.
- Learning and testing happen simultaneously.

Drawbacks

- Features may be misunderstood as bugs.
- Time consuming.
- Some bugs may be missed due to lack of requirements.

Adhoc Testing

- Testing application randomly without any test cases or any business requirement document.
- Adhoc testing is an informal testing type with an aim to break the system.
- Tester should have knowledge of application even thou he doesn't have requirements/test cases.
- This testing is usually an unplanned activity.



Monkey/Gorilla Testing

- Testing application randomly without any test cases or any business requirement document.
- Adhoc testing is an informal testing type with an aim to break the system.
- Tester do not have knowledge of application
- Suitable for gaming applications.

Adhoc Testing Vs Monkey Testing Vs Exploratory Testing

Adhoc Testing	Monkey Testing	Exploratory Testing
No Documentation	No Documentation	No Documentation
No Plan	No Plan	No Plan
Informal testing	Informal testing	Informal testing
Tester should know Application functionality	Testers doesn't know Application functionality	Testers doesn't know Application functionality
Random Testing	Random Testing	Random Testing
Intension is to break the application/find out corner defects	Intension is to break the application/find out corner defects	Intension is to learn or explore functionality of application
Any Applications	Gaming Applications	Any Applications which is new to tester

Positive Testing

- Testing the application with **valid inputs** is called as Positive Testing.
- It checks whether an application behaves as expected with positive inputs.
- For example -

Enter Only Numbers

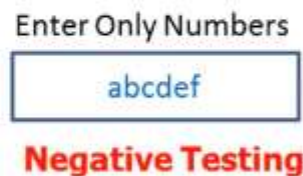
Positive Testing

There is a text box in an application which can accept only numbers. Entering values up to **99999** will be acceptable by the system and any other values apart from this should not be acceptable.

To do positive testing, set the valid input values from 0 to 99999 and check whether the system is accepting the values.

Negative testing

- Testing the application with **invalid inputs** is called as Negative Testing.
- It checks whether an application behaves as expected with the negative inputs.
- For example -



Negative testing can be performed by entering characters A to Z or from a to z. Either software system should not accept the values or else it should throw an error message for these invalid data inputs.

End-to-End Testing

“End-to-End Testing is performed to verify the overall functionality of the application by testing all the modules together from start to end.”

Simple Example:

In an eCommerce application:

- User registration
- Login
- Search product
- Add to cart
- Payment
- Order confirmation

Entire flow ni test chestharu.

Simple Example:

In an eCommerce application:

- User registration
- Login
- Search product
- Add to cart
- Payment
- Order confirmation

Entire flow ni test chestharu.

Simple Example:

In an eCommerce application:

- User registration
- Login
- Search product
- Add to cart
- Payment
- Order confirmation

Entire flow ni test chestharu.

Simple Example:

In an eCommerce application:

- User registration
- Login
- Search product
- Add to cart
- Payment
- Order confirmation

Entire flow ni test chestharu.

Localization Testing

“Localization Testing is performed to check whether the application works correctly for a specific language or region.”

Globalization Testing

“Globalization Testing is performed to check whether the application supports multiple languages and regions.”

Test design/Test case design/Test data design techniques

Used to prepare data for testing

Main aim for these techniques is to reduce data and more coverage

Test Design Techniques

Test Design Techniques are methods used to create effective test cases to find maximum defects with minimum test cases.

Equivalence Partitioning

“Equivalence Partitioning is used to divide input data into valid and invalid groups and test one value from each group.”

Boundary Value Analysis

“Boundary Value Analysis is used to test minimum and maximum limit values.”

Decision Table Testing

“Decision Table Testing is used to test different combinations of conditions and results.”

State Transition Testing

“State Transition Testing is used to test how the application changes from one state to another based on user actions.”

Error Guessing

“Error Guessing is used to find defects based on tester experience and assumptions.”

STLC

“STLC is a step-by-step process followed by the testing team to check whether the application is working correctly according to the requirements.”

Phases of STLC

1. Requirement Analysis

Testing team analyzes requirements and identifies:

- **What to test** → Identify which features need testing.
 - **Testable requirements** → Check whether requirements can be tested or not.
 - **Test scope** → Decide what is included and excluded in testing.
-

2. Test Planning

Test manager prepares:

- Test plan
 - Testing strategy
 - Resources
 - Schedule
 - Estimation
-

3. Test Case Development

Testers create:

- Test cases
 - Test data
-

4. Test Environment Setup

Required hardware and software environment is prepared for testing.

Example:

- Server setup
 - Database setup
 - Browser setup
-

5. Test Execution

Test cases are executed.

- Pass/Fail status checked
 - Defects identified
-

6. Defect Reporting & Tracking

Bugs are reported to developers and tracked until fixed.

7. Test Closure

Testing process is completed and reports are prepared.

Includes:

- Test summary report
- Lessons learned
- Testing completion status

Test Plan is a document that explains how testing will be done in a project, including scope, strategy, resources, schedule, and responsibilities.

Use Case

A Use Case describes how a user interacts with the application to achieve a goal.

Test Scenario:

“A Test Scenario is a high-level statement that describes what functionality needs to be tested.”

Test Case

A Test Case contains detailed test steps, test data, and expected results to verify a functionality.

Easy Flow

Use Case → Test Scenario → Test Case

Example:

1. Use Case

“User wants to buy a product.”

2. Test Scenarios

- * Test product search
- * Test add to cart
- * Test payment process

3. Test Cases

- * Enter product name in search box → Products should display
- * Click “Add to Cart” → Product should add successfully
- * Enter valid card details → Payment should succeed

One-line Definitions

- * Use Case → User activity/business flow
- * Test Scenario → What to test

* Test Case → How to test with steps and expected result

* Use Case → Prepared by Business Analyst (BA)

* Test Case → Prepared by Test Engineer / QA Tester

Test Suite

Test suite is group of test cases which belong to same category

Login Test Suite

It contains multiple test cases related to login functionality:

1. Verify login with valid username and password
2. Verify login with invalid password
3. Verify login with empty username
4. Verify “Forgot Password” functionality
5. Verify password visibility option

All these test cases together are called a Test Suite.

Bug Life Cycle

Bug Life Cycle is the journey of a bug from finding the defect to fixing and closing it. Stages of Bug Life Cycle:

New – Tester finds and reports the bug.

Assigned – Bug is assigned to a developer.

Open – Developer starts working on it.

Fixed – Developer fixes the bug.

Retest – Tester checks the fix.

Verified – Tester confirms the bug is fixed.

Closed – Bug is completely closed.

Other Possible States:

Rejected – Not a valid bug.

Duplicate – Same bug already reported.

Deferred – Fix postponed to future release.

Reopened – Bug still exists after fixing.

Requirement Traceability Matrix (RTM)

A Requirement Traceability Matrix (RTM) is a document used to check whether all requirements are covered by test cases.

It helps testers track:

requirements,
related test cases,
and testing status.

Test Environment

A Test Environment is the setup where testing is performed.

It includes:

hardware,
software,
database,
network,
and testing tools needed to test the application.

It also called as test bed

Test Execution

“Test execution is the process of running test cases and checking whether the application is working correctly or not.”

Activities:

- Execute test cases
- Compare actual and expected results
- Identify and report bugs
- Retest fixed bugs
- Update test reports

Inputs:

- Test plan
- Test cases
- Test data
- Requirement documents
- Build and environment

Outputs:

- Test results
- Bug reports

- Pass/Fail status
- Test summary report

Guidelines for Test Execution

“These are the basic rules followed while executing test cases during testing.”

Important Points:

- Testing should be done in QA environment
- Correct build should be used
- Test cases should be executed correctly
- Bugs should be reported properly
- Fixed bugs should be retested

Bug:

Defect send to developers by defect tracking tools like jira,quality center etc

Defect send to developers by defect tracking tools like jira,quality center etc

Defects are categorised by severity and priority

Severity

“Severity defines how serious the defect is and how much it affects the application functionality.”

Types:

- Critical
- Major
- Minor
- Low

Priority

“Priority defines how quickly the defect should be fixed.”

- Types:
- High
- Medium
- Low

Login button color wrong →

Severity = Low

Priority = Low

Application crashing →

Severity = High

Priority = High

→Severity means how serious the defect is, Priority means how urgent it is to fix.

Test Closure

Test closure is the final stage of STLC where we complete testing activities, prepare test reports, and formally close the testing process.

What happens in Test Closure?

All test cases are executed

Defects are closed or documented

Test summary report is prepared

Test metrics are collected

Lessons learned are recorded

Test Metrics:

Test metrics are values used to measure testing work.”

Examples of Test Metrics

- * Number of test cases executed
- * Number of test cases passed/failed
- * Defect count
- * Defect density
- * Test coverage
- * Testing progress percentage

Example

If 100 test cases are written and 80 are executed:

- * Executed = 80%
- * Remaining = 20%

These are called test metrics.

7 Principles of Software Testing

1. Early Testing

Testing should begin early to identify defects quickly and reduce cost.

2. Testing Shows Defects

Testing helps to find bugs in the application.

3. No Bug ≠ Successful Software

Even if there are no bugs, software fails if it does not satisfy customer needs.

4. Complete Testing is Not Possible

We cannot test the application in all possible ways.

5. Testing Depends on Project

Testing changes based on the type of application.

6. Same Testing Won't Find New Bugs

Using the same test cases repeatedly may not detect new defects.

7. More Bugs in Few Areas

Most defects are usually present in a few modules only.